G. Real Memory Management (OSNo08 ต้น)

> **📖 รู้จักส่วนนี้ก่อนอ่าน**
>
> **ความหมาย:** การจัดสรร RAM ให้ process ตอนโหลด (ยุคก่อนมี virtual memory) — จัดแบบช่องคงที่, ช่องแปรผัน, หรือแบ่งตามส่วน (Segmentation (การแบ่งหน่วยความจำตามส่วน)) พร้อมกลยุทธ์เลือกหลุม First/Best/Worst-fit (หยิบหลุมใหญ่ที่สุด)
>
> **ความสำคัญกับข้อสอบ:** โจทย์ first/best/worst-fit คำนวณเร็วได้คะแนนง่าย + Segmentation (เช็ค offset (ตำแหน่งเลื่อนภายในเพจ) < limit ก่อน แล้วค่อย base + offset) — และการแยก internal/external (ภายนอก (กระจายเป็นหลุม)) fragmentation (การแตกเป็นเศษพื้นที่) ให้ถูกตัว
>
> **ใช้จริงในคอมพิวเตอร์:** เป็นต้นแบบ memory allocator (malloc/free) ในทุกภาษา · ปัญหา fragmentation ยังพบจริงในระบบฝังตัว (embedded) และการจัดการหน่วยความจำ GPU

G1. Logical (เชิงตรรกะ (มองจากโปรแกรม)) vs Physical Address (ที่อยู่จริงใน RAM) + MMU

	Logical (Virtual) Address	Physical Address
สร้างโดย	CPU ปล่อยขณะรัน	ตำแหน่งจริงที่ memory unit มองเห็น
จุดอ้างอิง	โปรแกรมเริ่มที่ byte 0 เสมอ	เทียบจุดเริ่ม RAM จริง

MMU = ฮาร์ดแวร์แปลง logical (เชิงตรรกะ (มองจากโปรแกรม)) → physical (เชิงกายภาพ (ตำแหน่งจริง)) แบบง่ายสุดคือ**บวก Relocation (Base) Register**

`Physical Address = Relocation Register (รีจิสเตอร์เก็บตำแหน่งฐาน) + Logical Address (ที่อยู่เชิงตรรกะ (มองจากโปรแกรม))`

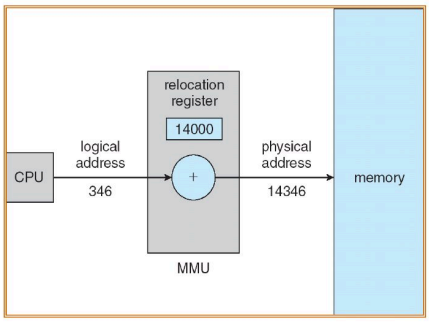
ตัวอย่างวิดีโอ: โปรแกรมโหลดที่ 14,000 · CPU ขอ byte 346 → PA = 14,000 + 346 = 14,346

Logical vs. Physical Address Space

- **Logical address** – generated by the CPU; also referred to as **virtual address**
- **Physical address** – address seen by the memory unit

สไลด์ p.3 Logical vs Physical (เชิงกายภาพ (ตำแหน่งจริง))

Dynamic relocation using a relocation register



สไลด์ p.5 Dynamic relocation ด้วย relocation register

G2. Static vs Dynamic Linking (มีตัวเลขให้คำนวณ)

- **Static linking**: ก๊อปโค้ด library รวมใน executable ตั้งแต่ compile — lib ซ้ำทุกโปรแกรม
- **Dynamic linking**: เลื่อนไป link (ลิงก์ (ทางเชื่อม)) ตอน run — ใส่แค่ **stub** (โค้ดจิ๋วหา library / แทนตัวเองด้วย address routine) → library โหลดครั้งเดียว แชร์กันทั้งระบบ (shared libraries)

`ตัวอย่างวิดีโอ: A(10MB) + B(20MB) + Lib(20MB)`
`Static: A=30MB, B=40MB → RAM 30+40 = 70MB (lib ซ้ำ 2 ชุด)`
`Dynamic: A=10+2, B=20+2, lib 1 ชุด → 12+20+22 = 54MB ประหยัด 16MB`

G3. Fixed Partition — Internal Fragmentation (เศษภายในช่องที่จองแล้ว)

- แบ่ง RAM ส่วน user เป็นช่องคงที่ล่วงหน้า → **มีกี่ช่องรับ process พร้อมกันได้เท่านั้น** · ป้องกันด้วย limit register (ขนาด) + relocation register (ตำแหน่ง): เช็ค LA < Limit ผ่านแล้วค่อย + Base
- **ตัวอย่างวิดีโอ:** Partitions P1=100K, P2=25K, P3=25K, P4=50K · Job1(30K)→P1 (เหลือ 70K ใช้ไม่ได้ = **internal frag**) · Job2(50K)→P4 พอดี · **Job3(30K) ต้องรอ!** เพราะ P2,P3 มีแค่ 25K แม้ว่างรวม 120K · Job4(25K)→P2 พอดี

🎯 คู่แฝด fragmentation

Fixed = Internal (เศษในช่องจองแล้ว ใครใช้ต่อไม่ได้) · **Variable + Segmentation = External** (เศษกระจายนอก รวมว่างเยอะแต่ช่องเดียวไม่พอ) · **Paging = ไม่มี External แต่มี Internal** ที่เพจสุดท้าย — ออกถูก/ผิดวนไป

G4. Variable Partition — External Fragmentation (เศษกระจายนอก (หลุมเล็ก ๆ)) + วิธีรักษา

จัดสรรตามขนาดจริง ตอนแรกไม่มีของเสีย แต่งานจบทิ้งช่องว่างกระจาย · **ตัวอย่าง:** หลุมว่าง 5+2+1 = 8MB แต่ขอ 7MB โหลดไม่ได้ (ช่องเดียวใหญ่สุด 5MB)

วิธีรักษา	ทำอะไร	ข้อจำกัด
Coalescing (การรวมหลุมว่างที่ติดกัน)	รวมหลุมที่**ติดกัน**เป็นหลุมใหญ่ 1 หลุม	หลุมไม่ติดกันก็ช่วยไม่ได้ — มักไม่พอ
Compaction (การอัดข้อมูลให้ติดกัน)	ลาก process มา**อัดติดกัน** รวมว่างปลายเดียว	overhead แพง และทำได้**เฉพาะ relocation แบบ dynamic (ตอน execution time)**

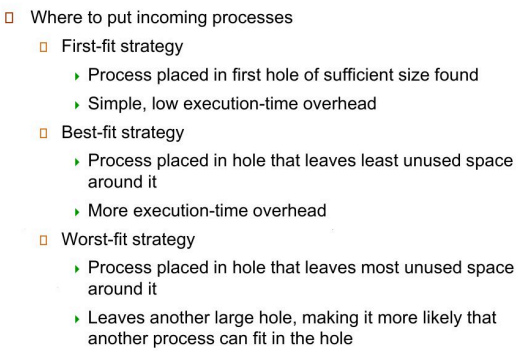
G5. First-fit (หยิบหลุมแรกที่พอ) · Best-fit (หยิบหลุมที่แน่นที่สุด) · Worst-fit 🔥 (ตัวอย่างจากวิดีโอ)

โจทย์ (สไลด์ p.21–24): หลุมเรียงตามลำดับใน RAM: **16, 14, 5, 30 MB** · process ใหม่ต้องการ **13 MB**

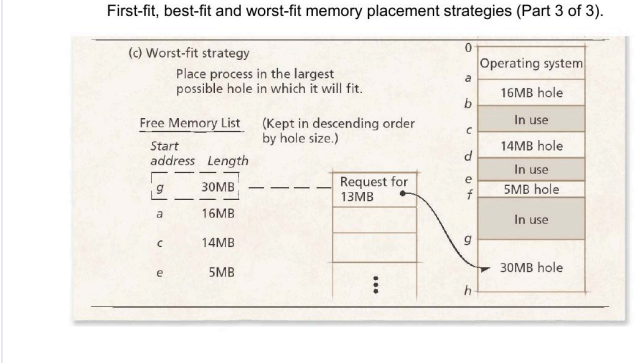
กลยุทธ์	หลักคิด	ลงหลุม	เศษเหลือ	จุดเด่น/ด้อย
First-fit	ไล่จากซ้าย เจอหลุมแรกที่ ≥ ขนาด เอาทันที	**16**	3	เร็วสุด (ไม่สแกนครบ) แต่เปลืองพื้นที่
Best-fit	สแกนครบ เลือกหลุมที่ "ใหญ่พอ + เหลือน้อยสุด"	**14**	1	คุ้มสุด แต่ช้า + ทิ้งเศษจิ๋วใช้ต่อไม่ได้
Worst-fit	สแกนครบ เลือกหลุมใหญ่สุด	**30**	17	ทิ้งชิ้นใหญ่ให้งานต่อไป (โดยรวมมักแพ้ best-fit)

🟢 เทคนิค 30 วินาที

เขียนเลขหลุมตามลำดับที่โจทย์ให้ (ห้ามจัดใหม่!) · First = หยุดหลุมแรกที่พอ · Best = เทียบ (หลุม−ขนาด) เลือกค่าน้อยสุด · Worst = หลุมใหญ่สุด · ถ้าโจทย์ให้จัดหลายงานต่อกัน อย่าลืมหักขนาดหลุมที่ใช้ไปแล้วด้วย



สไลด์ p.21 Memory Placement Strategies



สไลด์ p.24 ผลการจัดทั้ง 3 กลยุทธ์ (ตรงกับตารางข้างบน)

G6. Segmentation 🔥 — แบ่งตามสายตาโปรแกรมเมอร์

- แบ่งเป็น logical unit ขนาดไม่เท่ากัน: main, function, object, stack, global variables · Logical address = คู่ `<segment s, offset d>`
- Segment table แต่ละแถวมี **Base** (จุดเริ่มใน RAM) + **Limit** (ความยาว) · STBR ชี้ตาราง, STLR = จำนวน segment (s ใช้ได้ถ้า s < STLR)

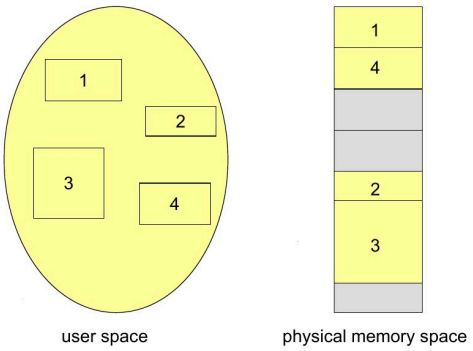
ขั้นแรก: เช็ค d < Limit — ถ้า d ≥ Limit → TRAP (error!) ทันที
ขั้นต่อมา: Physical Address = Base + Offset (ตำแหน่งเลื่อนภายในเพจ)

ตัวอย่างจากสไลด์ p.50: S1 (base 6300, limit 500) · S2 (base 4300, limit 400)

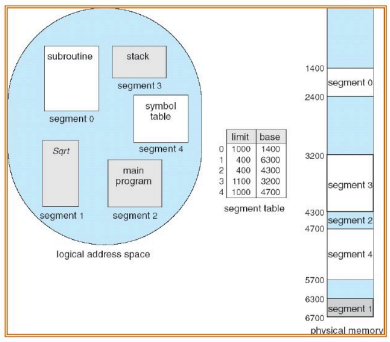
Logical	เช็ค	ผล
<1, 300>	300 < 500 ✓ → PA = 6300 + 300	**6600** ✓
<2, 400>	400 < 400 ✗ (400 ไม่ได้ "น้อยกว่า" 400)	**Trap / Error** ❌

🎯 จุดหลอก

เช็ค **ก่อนเสมอ แล้วค่อยบวก** — เคร่งครัดเรื่อง < ไม่ใช่ ≤ · Segmentation ยังเจ็บกับ **external fragmentation** (แต่ละ segment ต้องวางติดกันเป็นผืน) · สูตรอย่างสลับกับ paging (การแบ่งหน่วยความจำเป็นหน้า): Segment/Base = **Base + offset** / Paging = **frame (เฟรม (ก้อน RAM)) × size (ขนาด) + offset**



สไลด์ p.47 Logical view (user space) vs physical memory



สไลด์ p.50 Example of Segmentation (ตัวเลขตรงกับตาราง)